



API Integration

Practical Skills Module 5: Topic 1

[Visit our website](#)

Practical Skills Module 5: Topic 1

Topic Code	PM-05-PS01
Topic	Start using API with Python

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, the learner must be able to:

- PA0101 Get an API key
- PA0102 Test API endpoints
- PA0103 Retrieve information – GET
- PA0104 Add new data – POST
- PA0105 Change existing information – PUT
- PA0106 Delete existing information – DELETE
- PA0107 View the status codes of the response
- PA0108 Generate a Python snippet

Applied Knowledge

- AK0101 Concept, definition and functions of REST API

Internal Assessment Criteria

- IAC0101 Expected results with Python REST API are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-05



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario:

A **weather monitoring application** interacts with a weather API to fetch real-time weather data. In a real-world application, user preferences (such as preferred city, temperature units, contact method etc.) would typically be stored and managed on your own backend server, not through the weather API. Since we're not building a backend in this task, you will simulate user preference actions using a **mock API** such as JSONPlaceholder, which mimics real API behavior.

In this task, you will:

- Use the weather API (e.g., OpenWeatherMap) for retrieving weather data.
- Simulate adding, updating, and deleting user preferences using a mock API.
- Handle API response status codes appropriately in your application.
- Generate Python code snippets to automate these tasks.

Weather API Application

In this activity, you will build a Python program that connects to a weather API and simulates user preference management. This will give you practice with different HTTP request methods (GET, POST, PUT, DELETE) and handling API responses. Complete this activity in a Python file called **weather_api.py**

Steps:

1. **Connect to a Weather API using an API key**
 - Sign up at [WeatherAPI.com](https://www.weatherapi.com/) to obtain your free API key.
 - Use this key to authenticate your requests when retrieving weather data.
2. **Test the API connection with a sample city**
 - Choose a city such as **London** or **New York**.
 - Use the `/current.json` endpoint to request current weather data.

- Example:

```
http://api.weatherapi.com/v1/current.json?key=YOUR_API_KEY&q=London
```

3. Retrieve weather information using a GET request

- Write Python code that sends a GET request to the WeatherAPI endpoint.
- Parse the JSON response and display key weather details (e.g., temperature, condition).

4. Simulate saving user preferences with a POST request

- WeatherAPI is read-only, so use a mock API such as **JSONPlaceholder** to simulate saving preferences.
- Example endpoint: `https://jsonplaceholder.typicode.com/posts`
- Send a POST request with sample user preference data (e.g., preferred city, units).

5. Simulate updating preferences with a PUT request

- Use JSONPlaceholder again to send a PUT request that updates the user's preferences.
- Example endpoint: `https://jsonplaceholder.typicode.com/posts/1`

6. Simulate deleting preferences with a DELETE request

- Use JSONPlaceholder to send a DELETE request to remove the user's preferences.
- Example endpoint: `https://jsonplaceholder.typicode.com/posts/1`

7. Display the response status code after each API request

- Print the status code (e.g., 200 for success) to confirm whether the request was successful.

8. Include a reusable Python function for making requests

- Write a helper function (e.g., `make_request(method, url, data=None)`) that can handle GET, POST, PUT, and DELETE requests.
- Use this function throughout your program to avoid repeating code.



Take note

- Use **WeatherAPI.com** for the GET request to retrieve real weather data.
 - Use **JSONPlaceholder** for POST, PUT, and DELETE requests to simulate user preference management.
 - This combination ensures you apply all four HTTP methods (GET, POST, PUT, DELETE) as covered in the learning content.
 - You may use **London** or **New York** when testing the weather data request.
-



Code hack

You can find examples of how to use JSONPlaceholder for POST, PUT, and DELETE requests in the **KM5 Learner Guide (pages 30–31)**. JSONPlaceholder is a free online REST API designed for testing and practicing HTTP methods. Following the examples in the guide will help you structure your requests correctly.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
